

Security concerns #1026

Closed as not planned



duckdoom4 opened on Feb 14, 2023 · edited by duckdoom4

Edits ···

I have some concerns about the way some of this code is implemented.

To name the two I've noticed so far, the llm_math and sql_database chains.

It seems these two will blindly execute any code that is fed to it from the llm

This is a major security risk, since this opens anyone who uses these up for remote code execution. (The python one more than the sql one).

With a mitm attack, anyone can just return back a piece of code in the reply, pretending it is the bot. And if that's not enough, with some well crafted prompt, you can probably make it execute code as well (by making the llm return text with the same prompt pattern but custom python code)

I understand that this is in very early beta, but I've already seen this used in different places, due to ChatGPT's popularity.

In any case, it might be beneficial to switch from exec() to eval() for the python calculator, since eval() is build for the purpose of evaluating math expressions.

16

3



duckdoom4 on Feb 14, 2023 · edited by duckdoom4

Edits ··· Author

<https://blog.finxter.com/python-exec/>

This should give a general idea of how the exec function can be abused



 **blob42** mentioned this on Feb 14, 2023

✓ [Docker for development and sandboxing #1031](#)



phr33b4d on Feb 14, 2023 via email

···

I have a pull request in the adversarial-prompts repo of a prompt injection/code execution POC for Davinci.

The scenarios duckdoom4 posits are bona fide threats. SQL injection is the

number three attack on the most recent OWASP top ten. *See*

<https://owasp.org/www-project-top-ten/> (accessed Feb. 13, 2023). MITM

attacks are also a common vulnerability. *See *

https://owasp.org/www-community/attacks/Manipulator-in-the-middle_attack

(accessed Feb. 13, 2023).

Injection attacks (SQL and others) can be prevented via input sanitization*.

See*

<https://security.berkeley.edu/education-awareness/how-protect-against-sql-injection-attacks>

(accessed Feb. 13, 2023). SQL Injection vulnerabilities can be prevented

"in web applications by utilizing *parameterized database queries with bound, typed parameters and careful use of parameterized stored procedures in the database*". *Ibid*. (emphasis mine).

As soon as possible, I intend to do a code review with these issues in mind. I will certainly do so before I utilize these features. I would encourage anyone else with infoSec experience to have a look, and feel free to reach out with any thoughts.

...

2

kjannette on Feb 14, 2023

@duckdoom4 @hwchase17 please see [hwchase17/adversarial-prompts#7](#) (referenced in post immediately above). There is a serious underlying vulnerability here: both in the models themselves and in add-on frameworks (i.e. Langchain) that open up another attack vector.

Containerization via Docker is a good start, for those willing and able to use it. See [#1031](#)

But it fails to address the underlying vulnerability. Also, Docker escapes are very possible, so that is at best a band-aid. See <https://attackdefense.pentesteracademy.com/listingnoauth?labtype=container-security-container-host-security&subtype=container-security-container-host-security-breakouts>.

1

blob42 on Feb 15, 2023

Contributor

@kjannette I am putting together in [#1031](#) the band-aid solution using a non privileged docker user. I will follow up with a proposal update to the repls, bash and sql chains so they can execute in the container context when docker is setup.

I agree that the security risks are a big concern. My use of langchain will require heavy usage of shell contexts which I had to postpone until there is some minimal isolation.

Right now anyone pulling the repo and using the Repl/Shell/Sql chains is running arbitrary and potentially malicious code without any disclaimer.

May be adding a warning message that can be turned off by users so at least they are aware of the risks ?

1

2



brandco on Feb 16, 2023

...

Would running tools in docker containers solve some of the security issues?

It would be pretty easy to make a template out of a command like this

```
docker run --name python_langchain -it --rm python python -c
"print({python_expression_to_be_evaluated})"
```

A docker tool might be a nice addition to langchain for other use cases as well.



duckdoom4 on Feb 17, 2023 · edited by duckdoom4

Edits ▾

Author

...

Would running tools in docker containers solve some of the security issues?

It would be pretty easy to make a template out of a command like this

```
docker run --name python_langchain -it --rm python python -c
"print({python_expression_to_be_evaluated})"
```

A docker tool might be a nice addition to langchain for other use cases as well.

Well, it might help. But I don't think it's going to solve the core problem. Which is that

`{python_expression_to_be_evaluated}` could contain unsafe code/ code with mal intent.

For example; Imagine someone supplying the following python code: `)\nevil_python_function()` or even worse, if the command line string is not sanitized: `)"&[sys cmd to run]`

As mentioned before, simply swapping from `exec()` to `eval()` would already help a great deal (But eval is still dangerous), tho depending on the final intentions might be too limiting.

Another option is having some tool that sanitizes the allowed python code with eg. regex patterns



duckdoom4 on Feb 17, 2023 · edited by duckdoom4

Edits ▾

Author

...

There are plenty of useful sources online on how to do it safely and what to look out for. Here are some examples:

<https://stackoverflow.com/questions/10661079/restricting-pythons-syntax-to-execute-user-code-safely-is-this-a-safe-approach>

PyPy sandbox (I recommend using this):

<https://doc.pypy.org/en/latest/sandbox.html>

Example of how difficult this problem is:

<https://softwareengineering.stackexchange.com/questions/191623/best-practices-for-execution-of-untrusted-code/191628#191628>

2



blob42 on Feb 17, 2023 · edited by blob42

Edits ▾

Contributor

...

Quick update on the docker PR [#1055](#) . I made a fully isolate container with no elevated privileges. The dependencies and the langchain package are all compiled in the image.

I avoided the virtualenv (poetry, pyenv, venv ..) used a minimal python based image. When changed is detected only the the main package is installed.

Would be great to have some testers / feedback.

duckdoom4 on Feb 17, 2023

Author ...

@blob42 Check out PyPy's sandbox. This seems like a very well implemented sandboxing environment. Having that running in a docker container and we're basically as safe as can be.

<https://doc.pypy.org/en/latest/sandbox.html>

1

blob42 on Feb 17, 2023

Contributor ...

@duckdoom4 thanks I will read about it and see how to integrate in the container. I was thinking to have two separate images, one that can be used for dev/testing with live reload (bind mount) and an other one especially crafted for agents.

1

2

  **zachschi127** mentioned this on Feb 17, 2023

 Patch LLMMathChain exec vulnerability #1119

blob42 on Feb 18, 2023 · edited by blob42

Edits ▾ Contributor ...

I took a look to the `sandboxlib` project. I think should be feasible to use it for the Agent's container. However I noticed that there was no activity on the project since 3 years, the project's repo [is here](#)

There are other options that can be considered for running untrusted code. Here is a summary of what can be done on the sandboxing topic:

- Using a virtualized runtime for docker like [gVisor](#)
Pros: Almost same sandboxing as full virtualization
Cons: Performance ?
- Drop all capabilities from container [see](#)
- For Python: Use [sandboxlib](#)
- Update the exec family of Agent tools and allow execution on a remote shell (like ssh) . Users can redirect the shell to a full virtual machine (kvm, xen ...)

  **blob42** mentioned this on Feb 18, 2023

 docker dev and test environment #1055

duckdoom4 on Feb 18, 2023 · edited by duckdoom4

Edits ▾

Author



I took a look to the `sandboxlib` project. I think should be feasible to use it for the Agent's container. However I noticed that there was no activity on the project since 3 years, the project's repo [is here](#)

There are other options that can be considered for running untrusted code. Here is a summary of what can be done on the sandboxing topic:

- Using a virtualized runtime for docker like [gVisor](#)
Pros: Almost same sandboxing as full virtualization
Cons: Performance ?
- Drop all capabilities from container [see](#)
- For Python: Use [sandboxlib](#)
- Update the exec family of Agent tools and allow execution on a remote shell (like ssh) . Users can redirect the shell to a full virtual machine (kvm, xen ...)

Ah yes, you clicked to link to the old repo. This one is up to date :) : <https://foss.heptapod.net/pypy/pypy>

If you look at the 'how to' section on the page I mentioned, they posted that you should download from that link.

Tho RestrictedPython like used in [#1134](#) might also be good enough

All the other things you mentioned also sound like a great idea ☺

12 remaining items

Load more

jcchavez on Mar 21, 2023



There is a similar concern in Coreruleset by running user scripts in the host (see <https://github.com/SpiderLabs/ModSecurity/wiki/Reference-Manual-%28v3.x%29#user-content-exec>) that we are most likely tackling in [@corazawaf](#) by using WASM.

duckdoom4 on Mar 27, 2023

Author



Hi there from Wasm community, has anyone considered using Wasm as a sandboxed execution env for python repl? You can think of it as an isolated OS kernel in the userland (extremely lightweight) vs Docker containers sharing kernels with the host for example. See <https://til.simonwillison.net/webassembly/python-in-a-wasm-sandbox> for reference.

Let me know if it helps, and I would be more than happy to craft patches. Thanks!

This seems like an awesome solution and something that can be implemented to work out of the box. That's great for the average user that needs to be protected, but doesn't need the extra protection a docker environment would provide.



 **lieyanqzu** mentioned this [on Apr 17, 2023](#)

✓ [\[Feature\]: 希望可以集成LangChain框架，支持更多复杂场景下的插件API langbot-app/LangBot#409](#)

 **jupyter** mentioned this [on May 26, 2023](#)

🔒 [Issue: security concerns with `exec\(\)` via multiple agents and Shell tool #5294](#)

 **Jflick58** on May 28, 2023 · edited by Jflick58

Edits ▾ Contributor ⋮

[@hwchase17](#) [@dev2049](#) I'm planning on tackling a wasm compilation and execution solution to replace `exec`. For extra security, it may be worth dynamically scanning generated code and throwing errors if major vulnerabilities are flagged. At the least, it seems straightforward to me to replace `exec()` with wasm. I'm just checking that there's no other work in flight on this. FYI [#5294](#)

3

 **Jflick58** mentioned this [on Jun 3, 2023](#)

🔒 [RFC: Use wasm-exec package to sandbox code execution by the Python REPL tool #5388](#)

🔒 [Replace exec with wasm_exec #5640](#)

 **asimjalis** mentioned this [on Jun 9, 2023](#)

✓ [SQLDatabaseChain has SQL injection issue #5923](#)

 **timxielCN** on Jul 12, 2023

Hi guys - any update on security improvement? Perhaps `wasm compilation` per [#1026 \(comment\)](#)

 **aiakubovich** on Jul 15, 2023

any updates?

 **ssutharzan** on Jul 18, 2023 · edited by ssutharzan

Edits ▾ ⋮

Hi All, Does making a custom version of LangChain by removing code execution part break other functionalities of LangChain or create some other issues? I am thinking of using LangChain this way for now to minimise code execution vulnerabilities. LangChain is well modularized and could be possible map the inter-code dependencies to customize and install only the needed parts per usecase to be extra safe and easy to review. In this way more mature/safe parts of LangChain can be started to be used in production systems.



dosubot bot on Oct 19, 2023



Hi, [@duckdoom4](#)! I'm Dosu, and I'm helping the LangChain team manage their backlog. I wanted to let you know that we are marking this issue as stale.

From what I understand, you raised a concern about security issues with the `llm_math` and `sql_database` chains in the code. The suggestion was made to switch from `exec()` to `eval()` for the python calculator to mitigate this risk. There have been discussions about potential solutions such as using Docker containers, sandboxing with PyPy, RestrictedPython, and Wasm. Additionally, a pull request related to using RestrictedPython has been made.

Before we close this issue, we wanted to check if it is still relevant to the latest version of the LangChain repository. If it is, please let us know by commenting on the issue. Otherwise, feel free to close the issue yourself or it will be automatically closed in 7 days.

Thank you for your contribution and please don't hesitate to reach out if you have any further questions or concerns!



dosubot added stale on Oct 19, 2023



dosubot closed this as not planned on Oct 26, 2023



dosubot removed stale on Oct 26, 2023

[Sign up for free](#) to join this conversation on GitHub. Already have an account? [Sign in to comment](#)

Assignees

No one assigned

Labels

No labels

Type

No type

Projects

No projects

Milestone

No milestone

Relationships

None yet

 Code with agent mode

▼

  **Replace exec with wasm_exec**

langchain-ai/langchain

Participants



+8